LAPORAN PRAKTIKUM

POSTTEST (6)

ALGORITMA PEMROGRAMAN LANJUT

Disusun oleh:

Sabrina (2509106126)

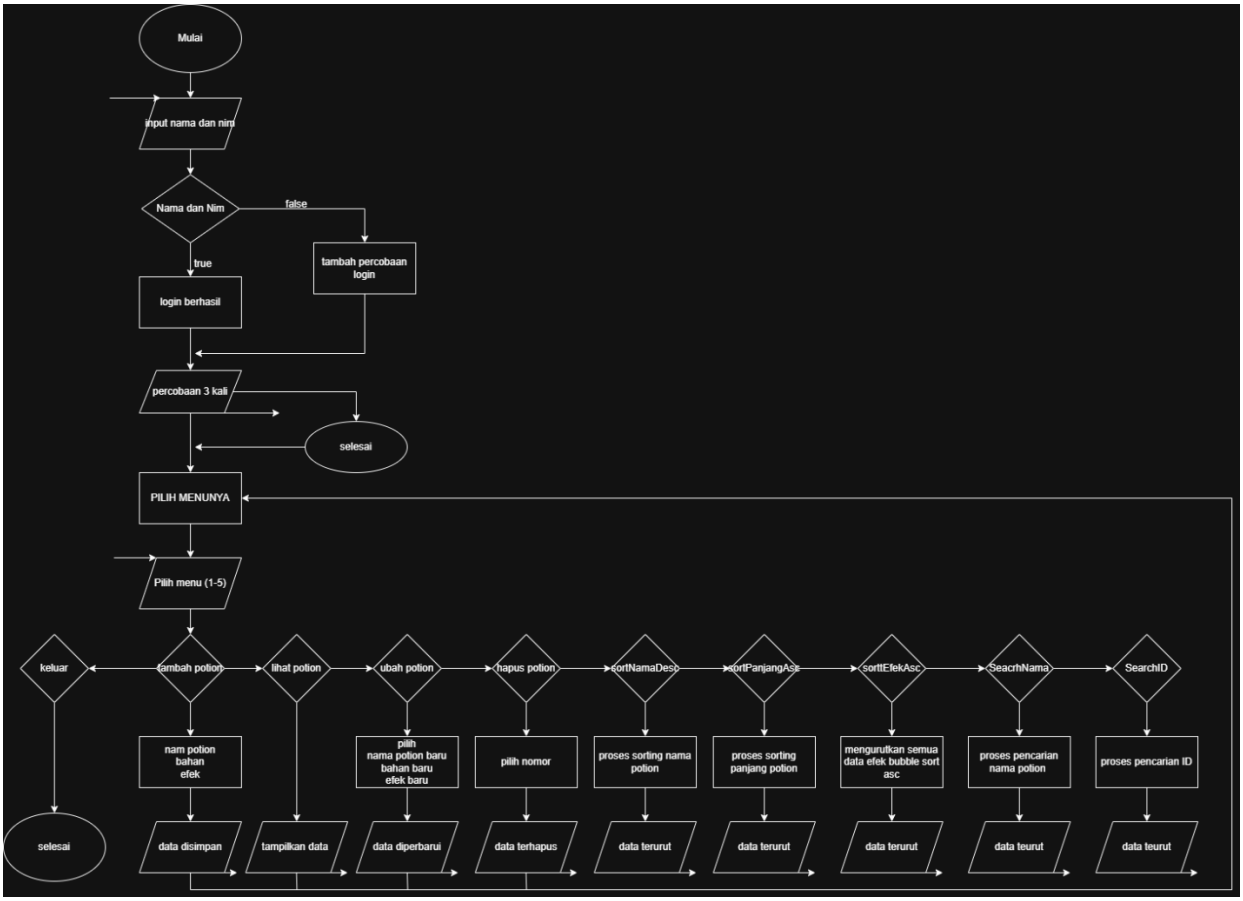
Kelas (C2 ‘25)

PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN SAMARINDA

2026

1. Flowchart



Gambar 1.1 flowchart dari program

2. Deskripsi Singkat Program

Program ini dibuat menggunakan bahasa pemrograman C++ untuk mengelola data potion dengan konsep CRUD (Create, Read, Update, Delete). Program dimulai dengan sistem login menggunakan nama sebagai username dan NIM sebagai password dengan maksimal 3 kali percobaan.

Data pada program disimpan menggunakan struct, yaitu User untuk menyimpan data pengguna dan Potion untuk menyimpan data potion yang terdiri dari nama potion, bahan, dan efek.

Setelah login berhasil, pengguna dapat mengakses menu utama yang berisi pilihan untuk menambah data potion, melihat data potion, mengubah data potion, dan menghapus data potion. Program akan terus berjalan menggunakan perulangan do-while sampai pengguna memilih menu keluar.

3. Source Code

SourceCode:

```
#include <iostream>
#include <string>
#include <cmath>
using namespace std;

struct User {
    string nama;
    string nim;
};

struct Potion {
    int id;
    string namaPotion;
    string bahan;
    string efek;
};

bool loginUser(User
*user) {
    string inputNama,
inputNim;
    int percobaan = 0;
```

```
    while (percobaan <
3) {
        cout << "===
LOGIN ===\n";
        cout << "Nama:
"; cin >> inputNama;
        cout << "NIM :
"; cin >> inputNim;

        if (inputNama ==
user->nama && inputNim
== user->nim) {
            cout <<
"Login berhasil!\n";
            return true;
        } else {
            cout <<
"Login gagal!\n";
            percobaan++;
        }
    }
    return false;
}

void tampil(Potion
*potion, int jumlah) {
    if (jumlah == 0) {
        cout << "Data
kosong!\n";
        return;
    }

    for (int i = 0; i <
jumlah; i++) {
        cout << i+1 <<
". "
```

```
            <<
potion[i].id << " | "
            <<
potion[i].namaPotion <<
" | "
            <<
potion[i].bahan << " | "
            <<
potion[i].efek << endl;
    }
}

void tambahPotion(Potion
*potion, int *jumlah) {
    cout << "\n===
TAMBAH POTION ===\n";

    cout << "Nama  : ";
    cin >>
potion[*jumlah].namaPoti
on;

    cout << "ID    : ";
    cin >>
potion[*jumlah].id;

    cout << "Bahan
tersedia: Glistering
Melon, Pufferfish,
Sugar\n";

    cout << "Bahan : ";
    cin >>
potion[*jumlah].bahan;

    cout << "Efek  : ";
```

```
    cin >>
potion[*jumlah].efek;

    (*jumlah)++;
    cout << "Potion
berhasil
ditambahkan!\n";
}

void hapusPotion(Potion
*potion, int *jumlah) {
    int x;
    cout << "Index: ";
cin >> x;

    if (x > 0 && x <=
*jumlah) {
        for (int i = x-
1; i < *jumlah-1; i++) {
            potion[i] =
potion[i+1];
        }
        (*jumlah)--;
        cout <<
"Berhasil dihapus!\n";
    } else {
        cout << "Data
tidak ditemukan\n";
    }
}

void bubbleSort(Potion
*potion, int jumlah) {
    for (int i = 0; i <
jumlah-1; i++) {
```

```
        for (int j = 0;
j < jumlah-i-1; j++) {
            if
(potion[j].namaPotion >
potion[j+1].namaPotion)
{
                Potion
temp = potion[j];

potion[j] = potion[j+1];

potion[j+1] = temp;
            }
        }
    }
    cout << "Bubble Sort
selesai!\n";
    tampil(potion,
jumlah);
}

void
selectionSort(Potion
*potion, int jumlah) {
    for (int i = 0; i <
jumlah-1; i++) {
        int minIndex =
i;
        for (int j =
i+1; j < jumlah; j++) {
            if
(potion[j].namaPotion <
potion[minIndex].namaPot
ion) {
                minIndex
= j;
```

```
            }
        }
        Potion temp =
potion[i];
        potion[i] =
potion[minIndex];
        potion[minIndex]
= temp;
    }
    cout << "Selection
Sort selesai!\n";
    tampil(potion,
jumlah);
}

void merge(Potion
*potion, int left, int
mid, int right) {
    Potion temp[100];
    int i = left, j =
mid+1, k = left;

    while (i <= mid && j
<= right) {
        if
(potion[i].namaPotion <=
potion[j].namaPotion)
            temp[k++] =
potion[i++];
        else
            temp[k++] =
potion[j++];
    }

    while (i <= mid)
temp[k++] = potion[i++];
```

```
    while (j <= right)
temp[k++] = potion[j++];

    for (int x = left; x
<= right; x++) {
        potion[x] =
temp[x];
    }
}

void mergeSort(Potion
*potion, int left, int
right) {
    if (left < right) {
        int mid = (left
+ right) / 2;

mergeSort(potion, left,
mid);

mergeSort(potion, mid+1,
right);
        merge(potion,
left, mid, right);
    }
}

void
linearSearchNama(Potion
*potion, int jumlah) {
    string cari;
    cout << "\nCari
Nama: ";
    cin >> cari;

    bool ketemu = false;
```

```
    for (int i = 0; i <
jumlah; i++) {
        if
(potion[i].namaPotion ==
cari) {
            cout <<
"Ditemukan: "
                <<
potion[i].id << " | "
                <<
potion[i].namaPotion <<
endl;
            ketemu =
true;
        }
    }

    if (!ketemu) cout <<
"Tidak ditemukan\n";
}

void sortID(Potion
*potion, int jumlah) {
    for (int i = 0; i <
jumlah-1; i++) {
        for (int j = 0;
j < jumlah-i-1; j++) {
            if
(potion[j].id >
potion[j+1].id) {
                Potion
temp = potion[j];

potion[j] = potion[j+1];
```

```
potion[j+1] = temp;
            }
        }
    }
}

void jumpSearchID(Potion
*potion, int jumlah) {
    int cari;
    cout << "\nCari ID:
";
    cin >> cari;

    sortID(potion,
jumlah);

    int step =
sqrt(jumlah);
    int prev = 0;

    while
(potion[min(step,
jumlah)-1].id < cari) {
        prev = step;
        step +=
sqrt(jumlah);

        if (prev >=
jumlah) {
            cout <<
"Tidak ditemukan\n";
            return;
        }
    }
```

```
    for (int i = prev; i
< min(step, jumlah);
i++) {
        if (potion[i].id
== cari) {
            cout <<
"Ditemukan: "
                <<
potion[i].id << " | "
                <<
potion[i].namaPotion <<
endl;
            return;
        }
    }

    cout << "Tidak
ditemukan\n";
}

int main() {
    User user = {"rina",
"126"};
    Potion potion[100];
    int jumlah = 0;

    if
(!loginUser(&user))
return 0;

    int menu;
    do {
        cout << "\n===
MENU ===\n";
        cout << "1.
Tambah\n";
```

```
        cout << "2.
Lihat\n";
        cout << "3.
Hapus\n";
        cout << "4.
Bubble Sort\n";
        cout << "5.
Selection Sort\n";
        cout << "6.
Merge Sort\n";
        cout << "7.
Linear Search (Nama)\n";
        cout << "8. Jump
Search (ID)\n";
        cout << "9.
Keluar\n";
        cout << "Pilih:
";
        cin >> menu;

        switch(menu) {
            case 1:
tambahPotion(potion,
&jumlah); break;
            case 2:
tampil(potion, jumlah);
break;
            case 3:
hapusPotion(potion,
&jumlah); break;
            case 4:
bubbleSort(potion,
jumlah); break;
            case 5:
selectionSort(potion,
jumlah); break;
```

```
            case 6:

mergeSort(potion, 0,
jumlah-1);
                cout <<
"Merge Sort selesai!\n";

tampil(potion, jumlah);
                break;
            case 7:
linearSearchNama(potion,
jumlah); break;
            case 8:
jumpSearchID(potion,
jumlah); break;
        }

    } while (menu != 9);

    cout << "Program
selesai.\n";
    return 0;
}
```

4. Hasil Output

```
=== SILAHKAN LOGIN ===
Masukkan Nama: rina
Masukkan NIM: 111
Login gagal!
=== SILAHKAN LOGIN ===
Masukkan Nama: rina
Masukkan NIM: 222
Login gagal!
=== SILAHKAN LOGIN ===
Masukkan Nama: rina
Masukkan NIM: 333
Login gagal!
Duhh gagal login 3 kali. Programnya berhenti.
```

Gambar 4.1 Output gagal login

```
=== SILAHKAN LOGIN ===
Masukkan Nama: rina
Masukkan NIM: 126
Login berhasil!
```

Gambar 4.2 Output login berhasil

```
=== SILAHKAN PILIH MENUNYA ===
1. Tambah Potion
2. Lihat Potion
3. Ubah Potion
4. Hapus Potion
5. Keluar
Pilih menu: 1

=== MENAMBAHKAN POTION ===
Masukkan Nama Potion: speed
Bahan potion: Glistering Melon, Pufferfish, Sugar
Masukkan Bahan Potion: sugar
Masukkan Efek Potion: Menambah_kecepatan
Potion berhasil ditambahkan!
```

Gambar 4.3 Output Tambah Potion

```
=== SILAHKAN PILIH MENUNYA ===
1. Tambah Potion
2. Lihat Potion
3. Ubah Potion
4. Hapus Potion
5. Keluar
Pilih menu: 2

=== LIHAT POTION ===
1. Tampilkan semua potion
2. Pilih potion
Pilihan: 1

Potion ke-1
Nama  : speed
Bahan : sugar
Efek  : Menambah_kecepatan

=== SILAHKAN PILIH MENUNYA ===
1. Tambah Potion
2. Lihat Potion
3. Ubah Potion
4. Hapus Potion
5. Keluar
Pilih menu: 2

=== LIHAT POTION ===
1. Tampilkan semua potion
2. Pilih potion
Pilihan: 2
Masukkan nomor potion: 1
Nama  : speed
Bahan : sugar
Efek  : Menambah_kecepatan
```

Gambar 4.4 Output Lihat Potion

```
=== SILAHKAN PILIH MENUNYA ===
1. Tambah Potion
2. Lihat Potion
3. Ubah Potion
4. Hapus Potion
5. Keluar
Pilih menu: 3

=== UBAH POTION ===
Pilih potion yang ingin diubah: 1
Nama potion baru: Water_Breathing
Bahan potion: Glistering Melon, Pufferfish, Sugar
Bahan baru yang dibutuhkan: Pufferfish
Efek baru: Bernapas_Dalam_Air
Data potion berhasil diubah!
```

Gambar 4.5 Output Ubah Potion

```
=== SILAHKAN PILIH MENUNYA ===
1. Tambah Potion
2. Lihat Potion
3. Ubah Potion
4. Hapus Potion
5. Keluar
Pilih menu: 4

=== MENGHAPUS POTION ===
Pilih potion yang ingin dihapus: 1
Potion berhasil dihapus!
```

Gambar 4.6 Output Hapus Potion

```
=== MENU ===
1. Tambah
2. Lihat
3. Hapus
4. Bubble Sort
5. Selection Sort
6. Merge Sort
7. Linear Search (Nama)
8. Jump Search (ID)
9. Keluar
Pilih: 4
Bubble Sort selesai!
1. 2 | Swiftness | Sugar | Menambah_kecepatan
2. 1 | Water_Breathing | Pufferfish | bernapas_dalam_air
3. 3 | healing | Glistering_Melon | Menyembuhkan_Darah
```

Gambar 4.7 Output bubble sort

```
=== MENU ===
1. Tambah
2. Lihat
3. Hapus
4. Bubble Sort
5. Selection Sort
6. Merge Sort
7. Linear Search (Nama)
8. Jump Search (ID)
9. Keluar
Pilih: 5
Selection Sort selesai!
1. 2 | Swiftness | Sugar | Menambah_kecepatan
2. 1 | Water_Breathing | Pufferfish | bernapas_dalam_air
3. 3 | healing | Glistering_Melon | Menyembuhkan_Darah
```

Gambar 4.8 Output selection sort

```
=== MENU ===
1. Tambah
2. Lihat
3. Hapus
4. Bubble Sort
5. Selection Sort
6. Merge Sort
7. Linear Search (Nama)
8. Jump Search (ID)
9. Keluar
Pilih: 6
Merge Sort selesai!
1. 2 | Swiftness | Sugar | Menambah_kecepatan
2. 1 | Water_Breathing | Pufferfish | bernapas_dalam_air
3. 3 | healing | Glistering_Melon | Menyembuhkan_Darah
```

Gambar 4.9 Output merge sort

```
=== MENU ===
1. Tambah
2. Lihat
3. Hapus
4. Bubble Sort
5. Selection Sort
6. Merge Sort
7. Linear Search (Nama)
8. Jump Search (ID)
9. Keluar
Pilih: 7

Cari Nama: healing
Ditemukan: 3 | healing
```

Gambar 4.10 Output linear Searching

```
=== MENU ===
1. Tambah
2. Lihat
3. Hapus
4. Bubble Sort
5. Selection Sort
6. Merge Sort
7. Linear Search (Nama)
8. Jump Search (ID)
9. Keluar
Pilih: 8

Cari ID: 2
Ditemukan: 2 | Swiftness
```

Gambar 4.11 Output jump Searching

```
=== SILAHKAN PILIH MENUNYA ===
1. Tambah Potion
2. Lihat Potion
3. Ubah Potion
4. Hapus Potion
5. Keluar
Pilih menu: 5
Program selesai.
```

Gambar 4.12 Output Keluar

5. Langkah-langkah GIT

5.1 GIT Add

```
PS C:\Users\LENOVO\OneDrive\Documents\praktikum-APL> git add .
```

Fungsi dari git add adalah untuk nandan file yang mau kita siapin sebelum di commit.

5.2 GIT Commit

```
PS C:\Users\LENOVO\OneDrive\Documents\praktikum-APL> git commit -m "kelar pt 3"
[main 7d242b7] kelar pt 3
 3 files changed, 176 insertions(+)
 create mode 100644 praktikum-apl/post-test/post-test-apl-3/2509106126-Sabrina-PT-3.cpp
 create mode 100644 praktikum-apl/post-test/post-test-apl-3/2509106126-Sabrina-PT-3.exe
 create mode 100644 praktikum-apl/post-test/post-test-apl-3/2509106126-Sabrina-PT-3.png
```

Fungsi dari git commit adalah buat menyimpan perubahan yang udah kita tandain pakai git add ke dalam riwayart git.

5.3 GIT Push

```
PS C:\Users\LENOVO\OneDrive\Documents\praktikum-APL> git push -u origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 16 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 575.72 KiB | 4.17 MiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/rina0706/pratikum-apl.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Fungsi git push adalah untuk ngirim perubahan yang udah kita commit di repo lokal ke repo online, biar orang lain liat atau kerja bareng.